Université Tahri Mohamed de Béchar
Faculté des sciences exactes
Département des Mathématiques et Informatique
2ême Année Ingénieur Informatique

Matière:

Algorithmiques et complexité

Cours n° 02

Enseignant : Benyamina Ahmed

Matière: Algorithmiques et complexité

Programme de la matière :

I- Rappels

II- Notion de complexité algorithmique

1. Notation de Landau O(n)
2. Classification des algorithmes
3. Règles de calcul sur les O(n)
4. Temps d'exécution
5. Conclusion

III- Algorithmes de tri

IV- Les arbres

V- Les graphes

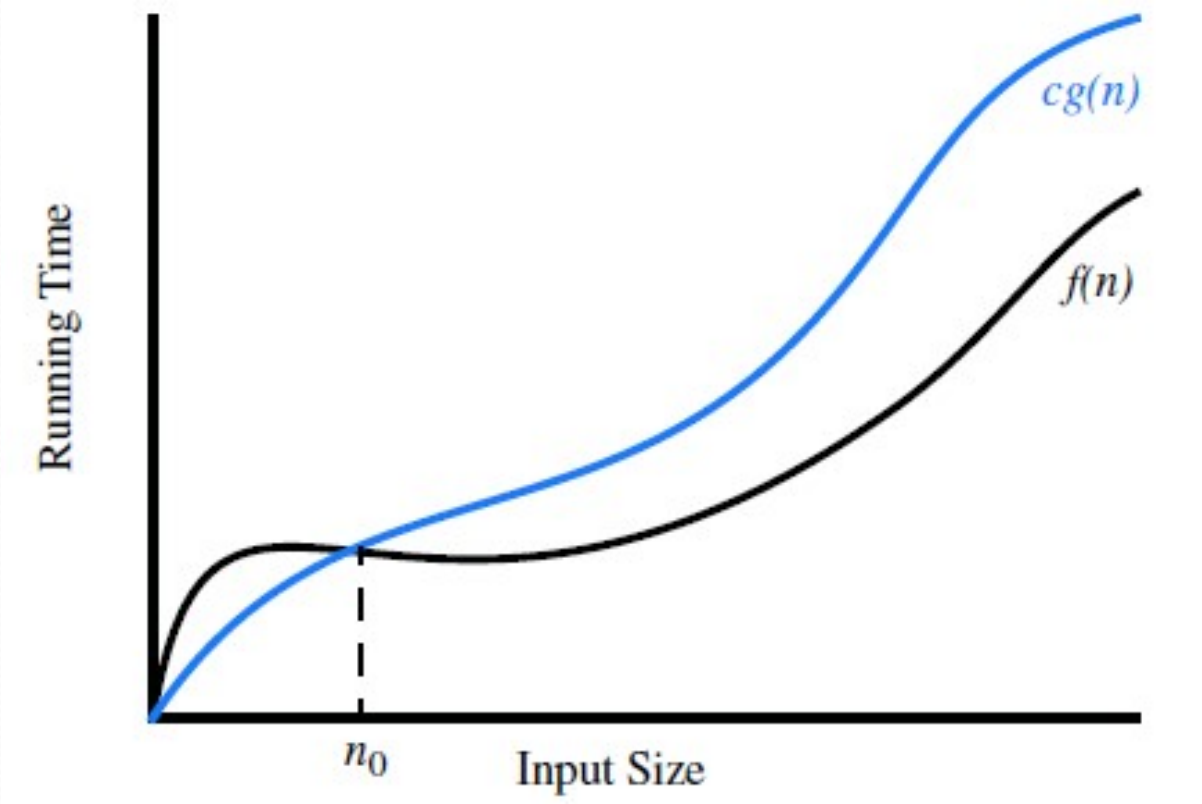
II. Notion de complexité algorithmique

1- Notation de Landau *O(n)* :

Hypothèse : La complexité dépend des données du problème

<u>Notation de Landau :</u> f est en O(g(n)) ou f ∈ O(g(n)) signifie :

$\forall$ $n>n_0\geq 0$, telle que $\exists c>0$ / $f(n)\leq c.g(n)$



II. Notion de complexité algorithmique

1- Notation de Landau *O(n)* :

Règle	Preuve
Si $f(n) \in O(kg(n))$, alors $f(n) \in O(g(n))$	Si $f(n) \in O(kg(n))$, alors $\exists k'>0$ tels que : $f(n) \leq kk'g(n)=\mathbf{K}g(n)$ $\forall n$ positif Alors **$f(n) \in O(g(n))$**
$5n^2+3n+4 \in O(n^2)$	On a: $\forall n>1$, $5n^2+3n+4< 5n^2+3n^2+4n^2 =12n^2$ Donc si on pose $k=12$ et $n_0=1$ On obtient : $\forall n>n0$, $5n^2+3n+4<kn^2$ Ce qui prouve que $5n^2+3n+4 \in O(n^2)$
2^{n+2} is $O(2^n)$.	$2^{n+2} = 2^n \cdot 2^2 = 4 \cdot 2^n$; par conséquent, nous pouvons prendre c = 4 et n0 = 1 dans ce cas.
si $f(n) = O(g(n))$ et $g(n) = O(h(n))$, alors $f(n) = O(h(n))$.	Ce que nous devons montrer ici, c'est que $f(n) \leq c3 \cdot h(n)$ pour $n > n3$ étant donné que $f(n) \leq c1 \cdot g(n)$ et $g(n) \leq c2 \cdot h(n)$, pour $n > n1$ et $n > n2$, respectivement. En mettant en cascade ces inégalités, on obtient ça $f(n) \leq c1 \cdot g(n) \leq c1c2 \cdot h(n)$ pour $n > n3 = \max(n1, n2)$.

II. Notion de complexité algorithmique

2- Classification des algorithmes

Cas d'utilisation courant	Cas d'utilisation courant
• Accéder au premier élément d'un ensemble de données	G(n)=1 ➔ C est O(1)
• Couper un ensemble de données en deux parties	G(n)=log(n) ➔ C est O(log(n))
• Algorithme traitant directement sur chacune des n donnés • Parcourir un ensemble de données	G(n)=n ➔ C est O(n)
• Couper répétitivement un ensemble de données en deux, et parcourir chacune de ces parties	G(n)=n.log(n) ➔ C est O(n.log(n))
• Algorithme traitant généralement des couples de données (dans deux boucles imbriquées)	G(n)=n^2 ➔ C est O(n^2)
• Complexité cubique. Idem quadratique mais avec ici par exemple trois boucles imbriquées	G(n)=n^3 ➔ C est O(n^3)
• Complexité exponentielle. Générer tous les sous-ensembles possibles d'un ensemble de données	G(n)=2^n ➔ C est O(2^n)
• Générer toutes les permutations possibles d'un ensemble de données	G(n)=n ! ➔ C est O(n !)

II. Notion de complexité algorithmique

3- Règles de calcul sur les O(n)

Règle	O(g(n))
1- $O(4n + 3)$	$O(n)$
2- $O(\log n) + O(\log n)$	$O(\log n)$
3- $O(n^2) + O(n) =$	$O(n^2)$
4- $O(m).O(n)=$	$0(m.n)$
5- $O(n^3).O(n^2.\log n) =$	$O(n^5n^2.\log n)$
6- $O(n^2/2) =$	$O(n^2) =$
7- $f(n)\in O(g(n))$ implique ($\Rightarrow$)	$g(n) \in O(f(n))$
8- $f(n)=O(g(n))$ implique	$\log(f(n))=O(\log(g(n)))$ où $\log(g(n))\geq 1\ \forall n >>0$
9- $f(n)=O(g(n))$ implique	$2^{f(n)}=O(2^{g(n)})$

II. Notion de complexité algorithmique

3- Règles de calcul sur les O(n)

Règle	O(g(n))
10- Si $f(n) = O(g(n))$ et $g(n) = O(h(n))$, alors	**$f(n) = O(h(n))$**
11- Si $f(n) \in O(kg(n))$ Alors $f(n) \in O(g(n))$	**c'est faux car $n^2 \in O(n^3)$ mais $n^3 \notin O(n^2)$**
12- Si f1(n)=O(g1(n)) et f2(n) = O(g2(n)) alors	(f1 + f2)(n) = O(max(g1(n), g2(n)))
13- Si f1(n)=O(g1(n)) et f2(n) = O(g2(n)) alors	**f1(n)f2(n) = O(g1(n) g2(n))**
14- Si f1(n)=O(g1(n)) et f2(n) = O(g2(n)) alors	**f1(n)f2(n) = O(g1(n) g2(n))**
15- la complexité d'un ensemble d'instructions	la somme de leurs complexités
16- Instruction `if`:	maximum entre le then et le else

II. Notion de complexité algorithmique

3- Règles de calcul sur les O(n)

Règle n° 17

Les opérations élémentaires telle que l'affectation, test, accès à un tableau, opérations logiques et arithmétiques, lecture ou écriture d'une variable simple ... etc, sont :

en O(1)

Règle n° 18

Instructions de répétition

1. la complexité de la boucle **for** est calculée par la :

- complexité du corps de cette boucle

- multipliée par le nombre de fois qu'elle est répétée.

2. En règle générale, pour déterminer la complexité d'une boucle **while**, il faudra avant tout déterminer :

- le nombre de fois que cette boucle est répétée,

- ensuite le multiplier par la complexité du corps de cette boucle.

Règle n° 19

Procédure et fonction:

Leur complexité est déterminée par celui de leur corps. L'appel à une fonction est supposé prendre un temps constant en O(1)

II. Notion de complexité algorithmique

4- Temps d'exécution

Temps d'exécution :

n	10	100	1000
$n\log(n)$	33	664	10^4
n^2	100	10^4	10^6
n^3	10^3	10^6	10^9
2^n	10^3	10^{30}	10^{300}
$\exp(n)$	2×10^4	10^{43}	10^{434}
$n!$	3.6×10^6	10^{158}	10^{2568}
$\log(n)$	3.3	6.6	10
$\sqrt{n}$	3.1	10	32

10^6 opérations élémentaires par seconde

Exemple de temps d'exécution

Plusieurs fonctions importantes

1 fonction constante, $f(x)=1$

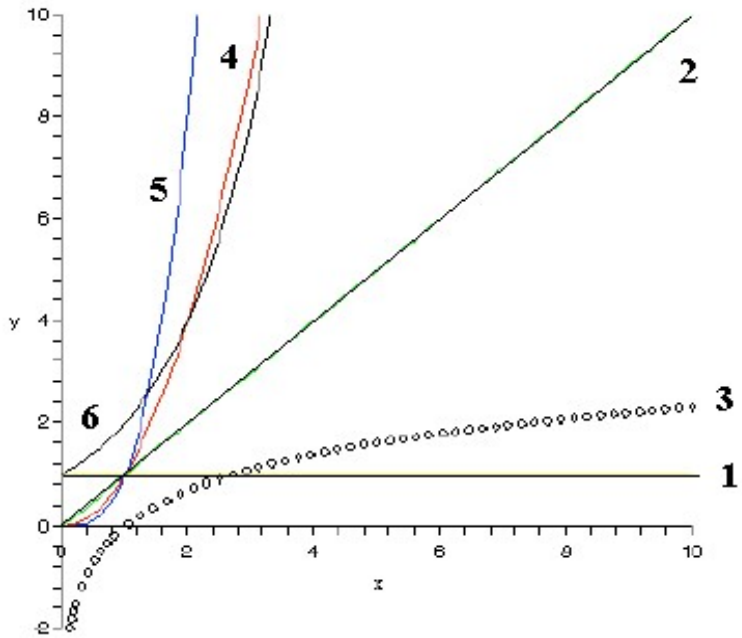
2 fonction linéaire, $f(x)=x$

3 fonction logarithmique, $f(x)=\log(x)$

4 fonction quadratique, $f(x)= x^2$

5 fonction cubique, $f(x)=x^3$

6 fonction exponential, $f(x)= 2^x$



II. Notion de complexité algorithmique

Conclusion

d'après ces schémas, il est clair que certains algorithmes sont utilisables pour résoudre des problèmes sur ordinateurs, et que d'autres ne sont pas, ou peu utilisable.

Les algorithmes utilisables pour des données de petite tailles sont ceux qui s'exécutent en temps : Constant, logarithmique, linéaire ou en n.log n.

Les algorithmes polynomiales en $O(n^n)$ $n>0$ ne sont pas vraiment utilisable pour $n>3$.

Les algorithmes exponentielles en $O(2^n)$ sont à peu près peu utilisable sauf pour des problèmes de petites taille.

Problème du tri des données

PROCHAIN COURS